Project - Low Level Design

on

Haven AI

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1.	Rudra Sthapak	EN22CS301828
2.	Rajvardhan Singh Rathore	EN22CS301786
3.	Ronak Bhawsar	EN22CS301824
4.	Rupendra Singh Tomar	EN22CS301833
5.	Raj Kumawat	EN22CS301779

Group Name: 04D7

Project Number: DO-27

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Shyam Patel

Academic Year: 2025-2026

Table of Contents

Chapter 1	Introduction	3
	1.1. Scope of the Document	3
	1.2. Intended Audience	3
	1.3. System Overview	3
Chapter 2	System Design	4-8
	2.1. Application Design	4
	2.2. Process Flow	5
	2.3. Information Flow	6
	2.4. Components Design	6
	2.5. Key Design Considerations	7
	2.6. API Catalogue	8
Chapter 3	References	9

1. Introduction

This document presents the Low-Level Design (LLD) of the Real Estate Support & Intelligent Triage Agent System. It provides detailed insights into system components, internal workflows, API structures, and deployment mechanisms. The system integrates Agentic AI capabilities with backend services and includes DevOps practices such as Docker-based containerization for consistent deployment.

1.1. Scope of the document

This document covers the internal design of the system including application modules, process flows, data movement, API interactions, and deployment strategies. It also includes the implementation of DevOps practices using Docker to automate application setup and execution.

1.2. Intended Audience

This document is intended for:

- Industry Mentor
- University Mentor
- Technical Review Panel (Datagami Members)
- Development Team Members
- System Architects

1.3. System overview

The system is an AI-powered real estate support agent that processes natural language queries using an Agentic AI approach. The system dynamically interprets user inputs, extracts relevant entities such as location, price, and property type, and performs filtered searches on a MongoDB database.

The backend is built using FastAPI, while AI orchestration is handled using LLM-based processing. The system is deployed using Docker containers to ensure consistency and scalability.

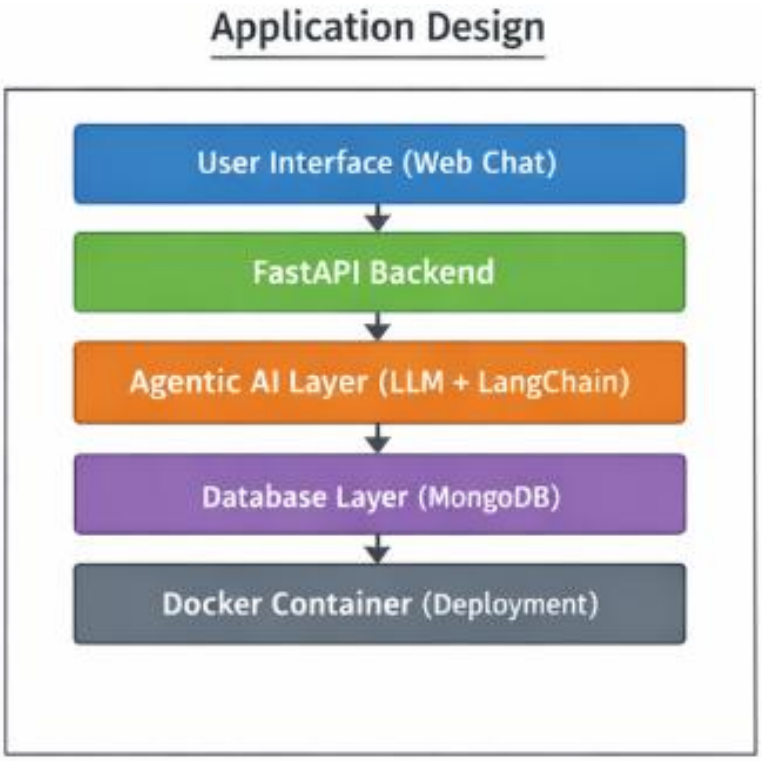
2. System Design

2.1. Application Design

The application follows a layered and modular architecture:

- **User Interface Layer**: Handles user interaction through chat interface.
- **Backend Layer (FastAPI)**: Processes API requests and business logic.
- **Agentic AI Layer**: Performs intent detection and entity extraction.
- **Database Layer (MongoDB)**: Stores structured data.
- **DevOps Layer (Docker)**: Handles containerized deployment.

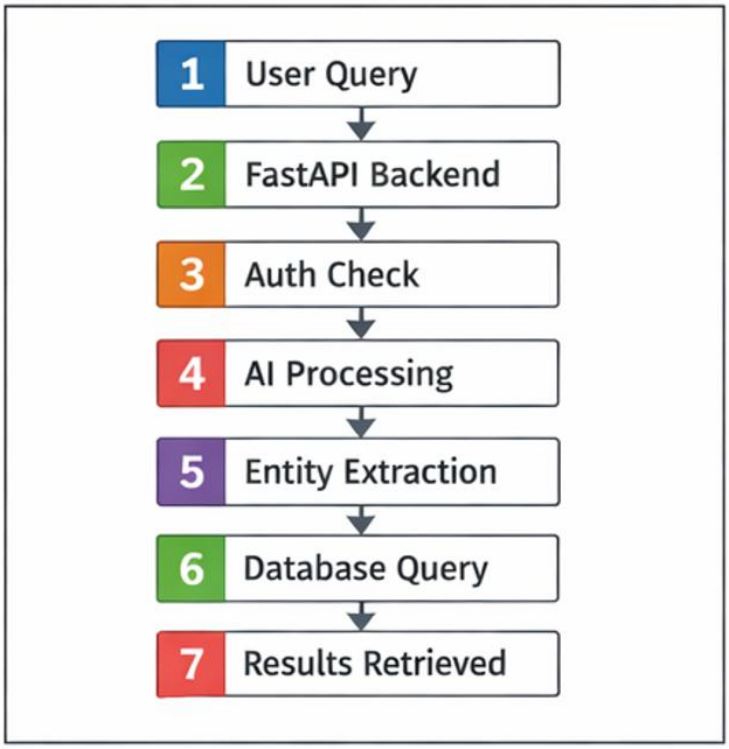
This design ensures scalability, modularity, and maintainability.



2.2. Process Flow

1. User submits query via chat interface.
2. Request is received by FastAPI backend.
3. Authentication is verified if required.
4. Query is processed by Agentic AI module.
5. Entities such as location and budget are extracted.

6. Backend generates filtered database query.
7. MongoDB returns relevant results.
8. Response is sent back to user.

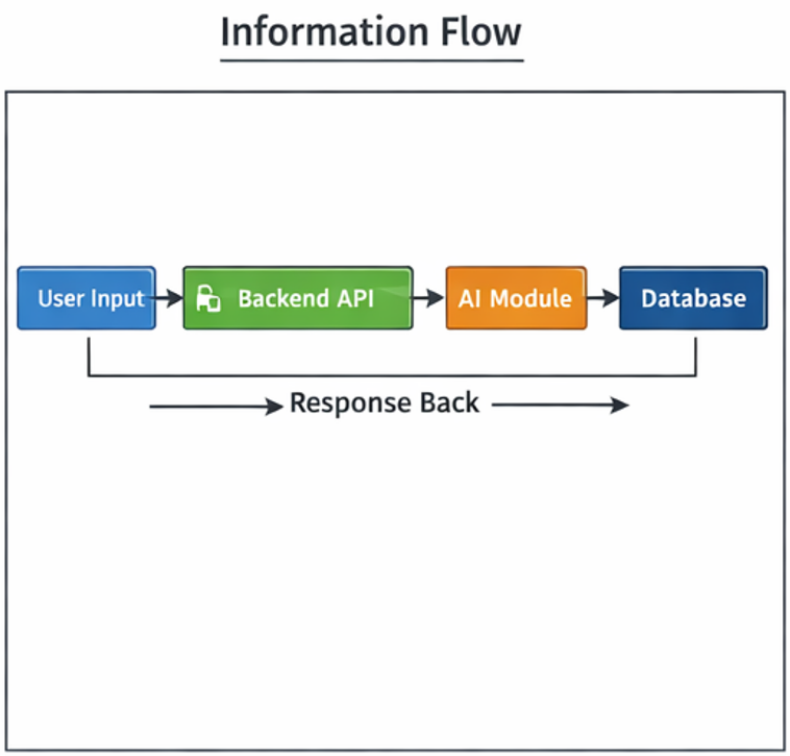


2.3. Information Flow

The system ensures structured data flow across components:

- User Input → Backend API
- Backend → AI Processing Layer
- AI → Query Generation
- Database → Backend Response
- Backend → User Interface

This flow ensures efficient communication between all modules.



2.4. Components Design

Authentication Component

Handles user registration, login, and OTP verification.

Chat Processing Component

Receives user queries and forwards them to AI module.

Agentic AI Component

Uses LLM to dynamically decide actions, extract entities, and call appropriate tools.

Property Search Component

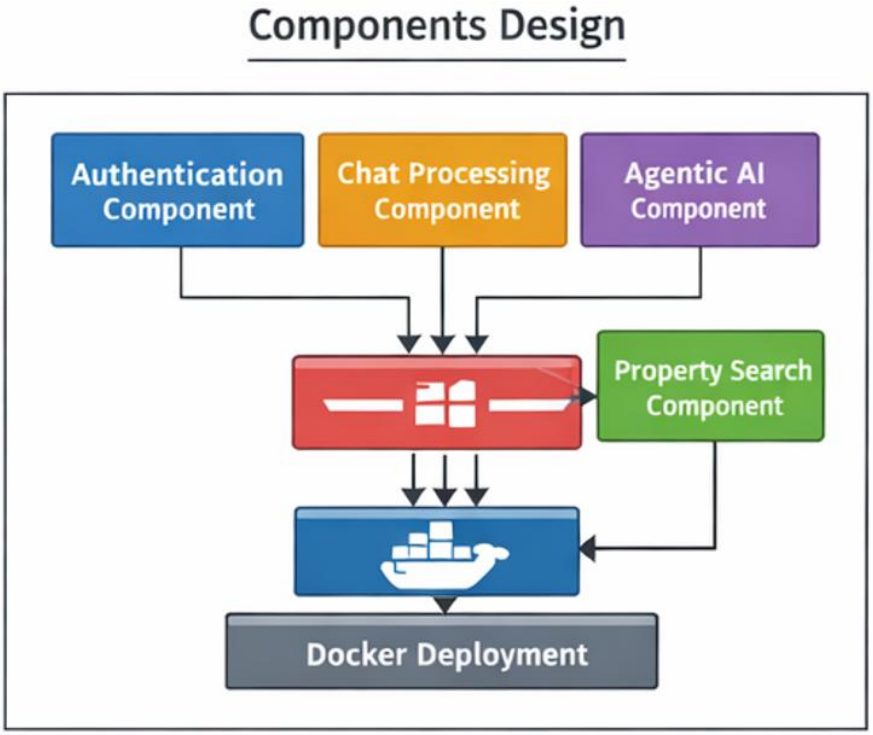
Executes filtered queries on MongoDB.

Database Component

Stores user data, property listings, and OTP records.

Docker Deployment Component

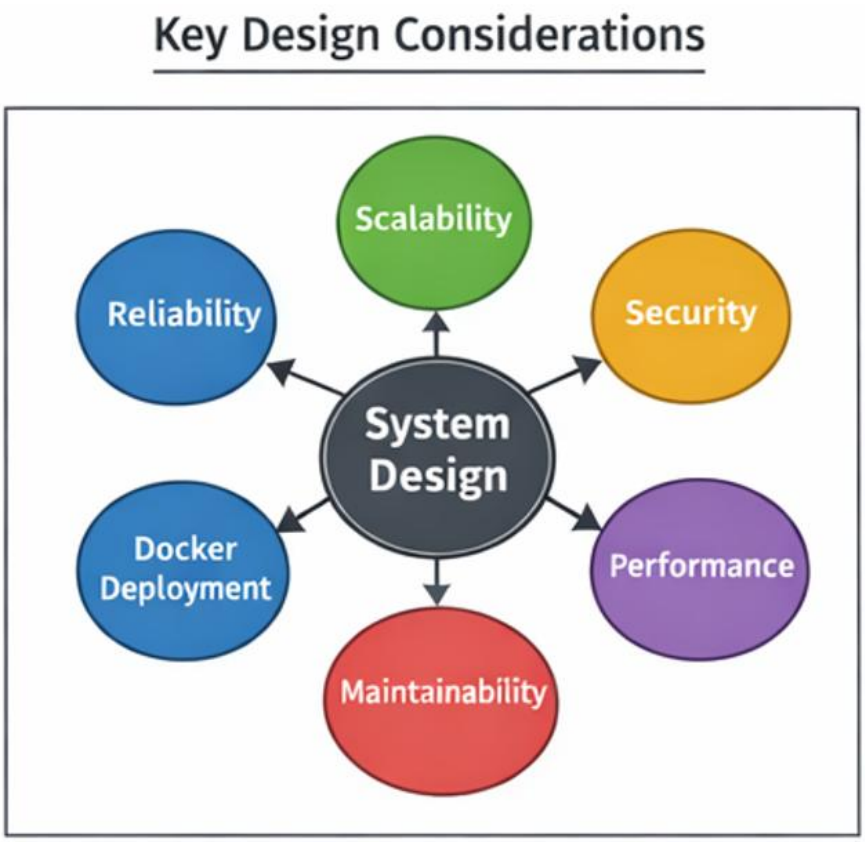
Packages the application into containers for consistent execution across environments.



2.5. Key Design Considerations

The system is designed with the following considerations:

- Modular and scalable architecture.
- Stateless API design.
- Secure authentication with OTP.
- Agentic AI-based dynamic processing.
- Asynchronous operations for performance.
- Containerized deployment using Docker.
- Reduced manual configuration using DevOps practices.



2.6. API Catalogue

Authentication APIs

- POST /register
- POST /login
- POST /verify-otp

Chat API

- POST /chat

Property APIs

- GET /properties
- GET /properties/filter

All APIs follow REST standards and return JSON responses.

3. References

- FastAPI Documentation. *FastAPI – Modern, Fast Web Framework for Building APIs with Python.*
Available at: https://fastapi.tiangolo.com/

- MongoDB Documentation. *MongoDB Official Documentation.*
Available at: https://www.mongodb.com/docs/

- Docker Inc. *Docker Documentation – Containerization Platform.*
Available at: https://docs.docker.com/

- LangChain Documentation. *LangChain Framework for LLM Applications.*
Available at: https://python.langchain.com/

- Google AI. *Generative AI & LLM Documentation.*
Available at: https://ai.google.dev/

- HashiCorp. *Terraform Documentation – Infrastructure as Code Tool.*
Available at: https://developer.hashicorp.com/terraform/docs

- Red Hat. *Ansible Documentation – Automation and Configuration Management.*
Available at: https://docs.ansible.com/

- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures (REST).* University of California, Irvine.